

Как усилить службу поддержки Eltera

Анализ под текущую архитектуру проекта и несколько вариантов развития — от «малой кровью» до полноценного help-desk

1 Что есть сейчас

Раздел «Поддержка» отправляет обращение в Telegram-бота (`app/services/support.py`). Есть три типа — **срочное / обычное / предложение**, у каждого свой шаблон, «приоритет» и «SLA» в тексте сообщения. К письму подставляются имя и e-mail из профиля. Если бот не настроен — обращение «принято», но не доставлено.

Сильные стороны

- Просто и работает: один POST — и команда видит обращение в Telegram.
- Уже различает тип обращения и подставляет отправителя из регистрации.
- Не роняет эндпоинт при сбое сети — обращение всё равно «принимается».

2 Где слабые места — именно для вашего продукта

Узкие места

- **Канал односторонний (fire-and-forget).** Обращение улетает в Telegram — и пользователь больше ничего не видит: ни номера тикета, ни статуса, ни ответа. «SLA в течение суток» — это просто текст, его никто не считает и не эскалирует.
- **Нет истории.** Обращения нигде не сохраняются (в коде прямо: «Записей в v1 нет»). Нельзя посмотреть свои прошлые вопросы, нельзя измерить нагрузку и качество поддержки.
- **Приоритет выбирает пользователь.** Все срочные — и всё «ПСРОЧНО». Классификацию и приоритет логичнее ставить автоматически по содержанию.
- **Нет контекста.** В тикет не попадает организация, тариф, роль, раздел/экран, версия, последние ошибки — поддержка переспрашивает «а что именно у вас не работает».
- **Кандидаты без поддержки.** Раздел «Поддержка» — только для авторизованного HR. Кандидат, у которого сломалась ссылка или упал тест на прохождении, оказывается в тупике — а это лицо продукта перед внешними людьми.
- **Не готово к мультиарендности.** Тикеты не привязаны к организации — при переходе на несколько компаний поддержку будет не разрулить по тенантам.

3 Варианты развития

Варианты не взаимоисключающие — это ступени. 1 и 2 дают самый большой скачок в качестве при малых усилиях и опираются на то, что у вас уже построено.

ВАРИАНТ 1 Замкнуть петлю: тикеты со статусом и ответом

ТРУДОЁМКОСТЬ

Низкая–средняя

ЭФФЕКТ НА UX

Очень высокий

СТОИМОСТЬ

≈ ₹0

Главная боль — канал односторонний. Делаем обращение настоящим тикетом: с номером, статусом и ответом, который возвращается пользователю в продукт. Без внешних сервисов.

Что делаем

- Таблица *support_tickets* + события (по образцу уже имеющейся *candidate_stage_events*): номер, статус «новый / в работе / решён», автор, организация.
- Раздел «Поддержка» → «Мои обращения»: список, статус, переписка по тикету.
- Двусторонний Telegram: **reply** в треде бота → webhook → запись ответа в тикет.
- Ответ пользователю — в Центр уведомлений (колокольчик) и письмом на e-mail.
- Реальный SLA: дедлайн по типу, просрочка → авто-эскалация (повторный алерт в Telegram).

Что переиспользуем (уже есть в проекте)

Telegram-бот, SMTP-почта (Reg.ru), Центр уведомлений, авторизация/организация, паттерн событий из адаптации и стадий кандидата — всё это в проекте уже есть.

Какую боль закрывает

Односторонний канал, отсутствие истории, «бумажный» SLA, нет связи с организацией.

ВАРИАНТ 2 AI-первая линия и база знаний (самообслуживание)

ТРУДОЁМКОСТЬ

Низкая

ЭФФЕКТ НА UX

Высокий

СТОИМОСТЬ

Копейки за обращение

У вас уже есть встроенный ИИ-ассистент с базой знаний о платформе. Превращаем его в первую линию: большинство вопросов он закрывает сам, в тикет уходит только то, что реально требует человека.

Что делаем

- Перед созданием тикета ассистент пытается ответить по своей базе знаний (KNOWLEDGE).
- Кнопка «Не помогло / позвать человека» → тикет с приложенной историей диалога.
- Раздел «База знаний / FAQ» из уже написанного контента ассистента — со ссылками на разделы.
- Подсказки по контексту: на экране ошибки — релевантная статья, а не пустая форма.
- Метка «отвечено ИИ / эскалировано» — чтобы видеть, сколько обращений снято автоматически.

Что переиспользуем (уже есть в проекте)

ИИ-ассистент (*app/services/assistant.py*) с готовой базой знаний и агент-циклом, дешёвая модель DeepSeek (~копейки за обращение).

Какую боль закрывает

Поток одинаковых вопросов в поддержку, долгое ожидание ответа на типовое, отсутствие FAQ.

ВАРИАНТ 3 Умные тикеты: контекст, авто-приоритет, метрики

ТРУДОЁМКОСТЬ

Средняя

ЭФФЕКТ НА UX

Высокий

СТОИМОСТЬ

Копейки за обращение

Поддержка тратит время на «а что у вас за экран и что не работает». Прикрепляем контекст автоматически и снимаем выбор приоритета с пользователя — его ставит модель по сути обращения.

Что делаем

- Авто-контекст в тикете: организация, тариф (биллинг Moneta), роль, текущий раздел/URL, версия, последние ошибки фронта/бэка.
- Авто-классификация и приоритет через AI (баг / вопрос / идея + срочность) — вместо ручного выбора.
- Дашборд поддержки: время первого ответа, время решения, доля решённых, поток по типам.
- Привязка к организации — основа для поддержки в мультиарендности и разбивки по тарифам.

Что переиспользуем (уже есть в проекте)

ИИ-ассистент для классификации, биллинг/тарифы (Moneta), модель организации и ролей, наблюдаемость (метрики AI-использования уже собираются).

Какую боль закрывает

Долгий разбор из-за нехватки контекста, ручной и неточный приоритет, нет управляемости по метрикам.

ВАРИАНТ 4 Поддержка для кандидатов и омниканальность

ТРУДОЁМКОСТЬ

Средняя–высокая

ЭФФЕКТ НА UX

Средний–высокий

СТОИМОСТЬ

₽0 (или подписка на виджет)

Кандидат — внешнее лицо продукта, но у него нет канала помощи во время прохождения теста. Закрываем этот разрыв и сводим все каналы в один поток тикетов.

Что делаем

- Лёгкая кнопка «Что-то не работает?» на экране прохождения оценки (без авторизации) — с авто-привязкой к ссылке/сессии теста.
- Входящая почта на support@ → автоматически создаёт/дополняет тикет (e-mail как канал).
- Единый поток: in-app, e-mail, Telegram — один тикет, один тред.
- Опционально — внешний виджет (Crisp / Usedesk / Chatwoot) вместо собственного UI, если нужен онлайн-чат.
- CSAT/NPS после закрытия тикета — короткая оценка качества поддержки.

Что переиспользуем (уже есть в проекте)

Поток прохождения оценки и оценочные ссылки, SMTP, накопленная модель тикетов из Варианта 1.

Какую боль закрывает

Кандидат в тупике при сбое теста, разрозненные каналы, нет замера удовлетворённости поддержкой.

4 Сравнение вариантов

Вариант	Трудоёмкость	Эффект	Стоимость	Когда брать
1. Замкнуть петлю	Низкая–средняя	Очень высокий	≈ Р0	Сразу — это фундамент
2. AI-первая линия + FAQ	Низкая	Высокий	Копейки	Сразу, параллельно с №1
3. Умные тикеты + метрики	Средняя	Высокий	Копейки	Когда поток вырастет
4. Кандидаты + омниканал	Средняя–высокая	Средний–высокий	Р0 / подписка	Перед массой внешних людей

5 Рекомендуемый путь

Поэтапно

- **Шаг 1 (сейчас).** Вариант 1 — тикеты со статусом и ответом + Вариант 2 — ИИ как первая линия. Оба почти не требуют новой инфраструктуры и сразу убирают «чёрную дыру», в которую улетают обращения.
- **Шаг 2 (к росту нагрузки).** Вариант 3 — авто-контекст, авто-приоритет и метрики поддержки; ложится на биллинг и мультиарендность, которые и так в планах.
- **Шаг 3 (перед массовым внешним трафиком).** Вариант 4 — поддержка кандидатов на прохождении и сведение каналов; критично, когда тесты начнут проходить реальные внешние люди.

Главная мысль: не нужно покупать внешний help-desk, чтобы заметно улучшить поддержку. У проекта уже есть всё для рывка — Telegram-бот, почта, центр уведомлений, ИИ-ассистент с базой знаний и модель событий. Самый дешёвый и самый заметный шаг — **замкнуть петлю обратной связи** (Вариант 1) и **поставить ИИ первой линией** (Вариант 2).

Eltera Assessment Intelligence · внутренний документ · подготовлено по текущему коду проекта